

COURSEWORK & STUDY GUIDE

Rust

Systems Programming

Every concept explained with code — ownership, lifetimes, traits, async, and beyond

Phase 1 through Phase 6 · Complete Reference

PHASE 1

Rust Fundamentals

1.1 Variables, Mutability & Shadowing

Rust is immutable by default. Every variable declared with `let` cannot be changed unless you explicitly opt in with `let mut`. This makes data flow predictable and prevents accidental mutation.

```
fn main() {
    // Immutable by default
    let x = 5;
    // x = 6; // error[E0384]: cannot assign twice to immutable variable

    // Opt into mutability with `mut`
    let mut count = 0;
    count += 1;
    count += 1;
    println!("Count: {count}"); // Count: 2

    // Shadowing - re-declare with the same name
    // Unlike mut, shadowing can change the type
    let spaces = "  "; // &str
    let spaces = spaces.len(); // usize - new variable, same name
    println!("Spaces: {spaces}"); // Spaces: 3

    // Constants - always immutable, must have a type, computed at compile time
    const MAX_POINTS: u32 = 100_000;
    println!("{MAX_POINTS}");
}
```

Shadowing vs mut

Shadowing creates a new variable with the same name. You can change the type. With `mut`, you can only change the value — the type is fixed. Use shadowing for transformations, `mut` for values you need to update repeatedly.

1.2 Data Types

Integer Types

Rust requires you to be explicit about integer sizes. There is no automatic promotion.

```
// Signed integers (can be negative)
let a: i8  = 127;    // -128 to 127
let b: i16 = 32_767;
let c: i32 = 2_147_483_647; // default for integer literals
let d: i64 = 9_223_372_036_854_775_807;
let e: i128 = 170_141_183_460_469_231_731_687_303_715_884_105_727;
let f: isize = -1; // pointer-sized (64-bit on 64-bit OS)

// Unsigned integers (non-negative only)
let g: u8  = 255;
let h: u32 = 4_294_967_295;
let i: u64 = 18_446_744_073_709_551_615;
let j: usize = 0; // used for indexing

// Integer literals in different bases
let decimal = 98_222;
let hex     = 0xff;
let octal   = 0o77;
let binary  = 0b1111_0000;
let byte: u8 = b'A'; // 65
```

Floating Point, Bool & Char

```
// Floating point - IEEE 754
let x: f64 = 3.14; // default, 64-bit double precision
let y: f32 = 2.0;  // 32-bit single precision

// Boolean
let t: bool = true;
let f: bool = false;

// char - a Unicode scalar value (4 bytes, not 1!)
let letter: char = 'z';
let emoji: char = '👍';
let kanji: char = '国';

// Compound: Tuple - fixed length, can mix types
let tup: (i32, f64, bool) = (500, 6.4, true);
let (x, y, z) = tup; // destructure
let first = tup.0;  // index access

// Compound: Array - fixed length, same type, stack-allocated
let arr: [i32; 5] = [1, 2, 3, 4, 5];
let zeroes = [0; 10]; // [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
let third = arr[2];    // 3 - zero-indexed
// arr[10];            // panics at runtime: index out of bounds
```

1.3 Ownership — The Core Concept

Ownership is Rust's answer to memory management. It replaces both garbage collection and manual malloc/free with a set of compile-time rules that the borrow checker enforces automatically.

The Three Ownership Rules

```
// RULE 1: Each value has exactly one owner
// RULE 2: There can only be one owner at a time
// RULE 3: When the owner goes out of scope, the value is dropped (freed)

fn main() {
    {
        let s = String::from("hello"); // s owns the string
        // s is valid here
    } // s goes out of scope - Rust calls drop(), memory is freed automatically
    // s is no longer valid here
}
```

Move Semantics

```
// Heap-allocated types (String, Vec, etc.) are MOVED on assignment
let s1 = String::from("hello");
let s2 = s1; // s1 is MOVED into s2 - s1 is now invalid

// println!("{s1}"); // error[E0382]: borrow of moved value: `s1`
println!("{s2}"); // OK

// Stack-only types (integers, bool, etc.) are COPIED
// because they implement the Copy trait
let x = 5;
let y = x; // x is COPIED - both x and y are valid
println!("{x} and {y}"); // 5 and 5

// To copy a heap type, use .clone() - an explicit deep copy
let s1 = String::from("hello");
let s2 = s1.clone(); // explicit deep copy - both s1 and s2 are valid
println!("{s1} and {s2}");
```

Ownership & Functions

```
fn takes_ownership(s: String) { // s comes in
    println!("{s}");
} // s is dropped here

fn makes_copy(x: i32) { // i32 is Copy, so a copy is made
    println!("{x}");
}

fn main() {
    let s = String::from("hello");
    takes_ownership(s); // s is MOVED into the function
    // println!("{s}"); // error - s was moved
}
```

```
let x = 5;
makes_copy(x);           // x is copied
println!("{}", x);       // fine - x is still valid

// To keep using a value after a function call: return it back
let s1 = String::from("hello");
let s2 = takes_and_gives_back(s1); // moved in, then moved out
println!("{}", s2);
}

fn takes_and_gives_back(s: String) -> String {
    s // return transfers ownership back to the caller
}
```

1.4 Borrowing & References

Moving values into every function is cumbersome. Borrowing lets you pass a reference to a value without transferring ownership. The borrow checker ensures references are always valid.

```
// &T - immutable (shared) reference
fn calculate_length(s: &String) -> usize {
    s.len() // we can read but not modify
} // s goes out of scope here, but the String it refers to is NOT dropped

fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1); // pass a reference - s1 is still owned here
    println!("Length of '{s1}' is {len}"); // s1 still valid!
}

// &mut T - mutable reference - allows modification
fn append_world(s: &mut String) {
    s.push_str(", world");
}

fn main() {
    let mut s = String::from("hello");
    append_world(&mut s);
    println!("{}", s); // hello, world
}
```

The Borrow Rules

```
// RULE: You can have EITHER
//   - Any number of immutable references (&T), OR
//   - Exactly ONE mutable reference (&mut T)
// But NEVER both at the same time.
```

```
let mut s = String::from("hello");

let r1 = &s; // OK - first immutable reference
let r2 = &s; // OK - second immutable reference
// let r3 = &mut s; // ERROR - cannot borrow as mutable while immutable refs exist

println!("{r1} and {r2}"); // r1 and r2 used here - their lifetime ends

let r3 = &mut s; // OK now - r1 and r2 no longer used
println!("{r3}");

// Dangling reference - Rust prevents this at compile time
// fn dangle() -> &String {
//     let s = String::from("hello");
//     &s // error: s is dropped at end of function, reference would dangle
// }
```

String Slices — &str

```
// &str is a reference to a portion of a String (or a string literal)
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &byte) in bytes.iter().enumerate() {
        if byte == b' ' {
            return &s[..i]; // return slice up to the space
        }
    }
    &s[..] // return the whole string if no space found
}

let sentence = String::from("hello world");
let word = first_word(&sentence); // works with both &String and &str
println!("{word}"); // hello

// String literals ARE &str - they're slices into the binary
let s: &str = "I am a string slice";
```

1.5 Functions & Control Flow

```
// Functions - typed parameters and return type
fn add(x: i32, y: i32) -> i32 {
    x + y // expressions don't need semicolons - last expr is the return value
}

// Statements vs expressions
// Statement: does not return a value; ends with ;
let y = 6;
// Expression: evaluates to a value; no semicolon
let z = { let a = 3; a * 2 }; // z = 6
```

```
// if is an expression – can be used in let
let condition = true;
let number = if condition { 5 } else { 6 };

// loop returns a value via break
let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 { break counter * 2; } // returns 20
};

// for loops use iterators – the idiomatic way to loop
let arr = [10, 20, 30, 40];
for element in arr {
    println!("{element}");
}

// Range-based for
for i in 0..5 { print!("{i} "); } // 0 1 2 3 4
for i in 0..=5 { print!("{i} "); } // 0 1 2 3 4 5
```

PHASE 2

Structs, Enums & Pattern Matching

2.1 Structs

Structs are named groups of related data. They are the primary way to create custom data types in Rust.

```
// Named-field struct
#[derive(Debug)] // auto-generate Debug printing
struct User {
    username: String,
    email: String,
    age: u32,
    active: bool,
}

fn main() {
    let mut user1 = User {
        username: String::from("alice"),
        email: String::from("alice@example.com"),
        age: 30,
        active: true,
    };

    user1.email = String::from("alice@new.com"); // mut required for field update

    // Struct update syntax - spread remaining fields from another instance
    let user2 = User {
        email: String::from("bob@example.com"),
        username: String::from("bob"),
        ..user1 // copy remaining fields from user1
        // NOTE: user1.email and user1.username are moved here
    };

    println!("{:?}", user2); // pretty-print with Debug
}

// Tuple struct - named type, positional fields
struct Point(f64, f64, f64);
struct Colour(u8, u8, u8);
let p = Point(1.0, 2.0, 3.0);
let c = Colour(255, 0, 128);
println!("{}", p.0, p.1, p.2);

// Unit struct - no fields, used as markers
struct AlwaysEqual;
```

Methods with impl


```

#[derive(Debug)]
struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    // Associated function (no self) - called like Rectangle::new()
    fn new(width: f64, height: f64) -> Self {
        Self { width, height } // field init shorthand when param names match
    }

    // Method - takes &self (immutable borrow of the instance)
    fn area(&self) -> f64 {
        self.width * self.height
    }

    fn perimeter(&self) -> f64 {
        2.0 * (self.width + self.height)
    }

    fn is_square(&self) -> bool {
        self.width == self.height
    }

    // Mutable method - takes &mut self
    fn scale(&mut self, factor: f64) {
        self.width *= factor;
        self.height *= factor;
    }
}

let mut r = Rectangle::new(10.0, 5.0);
println!("Area: {}", r.area()); // 50.0
println!("Square: {}", r.is_square()); // false
r.scale(2.0);
println!("{:?}", r); // width: 20.0, height: 10.0

```

2.2 Enums

Rust enums are far more powerful than enums in most languages. Each variant can hold different types and amounts of data. They are the foundation of Rust's error handling and null safety.

```

// Simple C-style enum
#[derive(Debug)]
enum Direction { North, South, East, West }

// Enum with data in variants
#[derive(Debug)]
enum Shape {
    Circle(f64), // one f64 (radius)
}

```

```

    Rectangle(f64, f64),          // two f64 (width, height)
    Triangle { base: f64, height: f64 }, // named fields
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle(r)          => std::f64::consts::PI * r * r,
            Shape::Rectangle(w, h)    => w * h,
            Shape::Triangle { base, height } => 0.5 * base * height,
        }
    }
}

let shapes = vec![
    Shape::Circle(5.0),
    Shape::Rectangle(4.0, 6.0),
    Shape::Triangle { base: 3.0, height: 8.0 },
];

for shape in &shapes {
    println!("{:?} -> area = {:.2}", shape, shape.area());
}

```

2.3 Option<T> — Safe Null

Option<T> replaces null. There are only two possibilities: Some(T) when a value exists, or None when it doesn't. The compiler forces you to handle both cases.

```

// Option<T> is defined in std as:
// enum Option<T> { Some(T), None }

fn find_first_even(numbers: &[i32]) -> Option<i32> {
    for &n in numbers {
        if n % 2 == 0 { return Some(n); }
    }
    None
}

let nums = vec![1, 3, 5, 8, 10];
match find_first_even(&nums) {
    Some(n) => println!("First even: {n}"),
    None    => println!("No even numbers"),
}

// Convenience methods on Option
let opt: Option<i32> = Some(42);

opt.unwrap();           // 42 - panics if None!
opt.unwrap_or(0);       // 42 - returns 0 if None
opt.unwrap_or_else(|| compute_default()); // 42 - calls fn if None
opt.expect("should have value"); // 42 - panics with custom message if None

```

```
// map - transform Some(T) to Some(U)
let doubled = opt.map(|x| x * 2); // Some(84)

// and_then - chain operations that return Option
let result = opt
    .map(|x| x + 1)           // Some(43)
    .and_then(|x| if x > 40 { Some(x) } else { None }) // Some(43)
    .unwrap_or(0);

// if let - concise single-arm pattern match
if let Some(value) = opt {
    println!("Got: {value}");
}
```

2.4 Result<T, E> — Explicit Errors

Result<T, E> is Rust's error handling mechanism. It forces you to acknowledge that an operation might fail. There is no hidden exception mechanism — errors are just values.

```
// Result<T, E> is defined as:
// enum Result<T, E> { Ok(T), Err(E) }

use std::num::ParseIntError;

fn parse_age(s: &str) -> Result<u32, ParseIntError> {
    let n: i64 = s.trim().parse()?; // ? propagates error if parse fails
    if n < 0 || n > 150 {
        // We'd normally define a custom error here
        // For brevity, we re-use the parse error pathway
        return "-1".parse::<u32>(); // will fail with ParseIntError
    }
    Ok(n as u32)
}

match parse_age("25") {
    Ok(age) => println!("Age: {age}"),
    Err(e)  => println!("Error: {e}"),
}

// The ? operator - propagate errors up the call stack
use std::fs;
use std::io;

fn read_username_from_file() -> Result<String, io::Error> {
    // Without ?: match and propagate manually
    // let content = match fs::read_to_string("hello.txt") {
    //     Ok(s)    => s,
    //     Err(e) => return Err(e),
    // };
```

```
// With ?: propagate error, continue if Ok
let content = fs::read_to_string("hello.txt")?;
Ok(content.trim().to_string())
}

// Result convenience methods
let r: Result<i32, &str> = Ok(42);
r.unwrap();           // 42 - panics on Err
r.unwrap_or(0);       // 42 - returns 0 on Err
r.map(|x| x * 2);      // Ok(84)
r.map_err(|e| format!("{}", e)); // transform the error type
r.is_ok();             // true
r.is_err();            // false
```

Custom Error Types with thiserror

```
// thiserror crate - ergonomic custom errors
// Add to Cargo.toml: thiserror = "1"
use thiserror::Error;

#[derive(Debug, Error)]
enum AppError {
    #[error("IO error: {0}")]
    Io(#[from] std::io::Error),

    #[error("Parse error: {0}")]
    Parse(#[from] std::num::ParseIntError),

    #[error("Validation failed: {field} - {message}")]
    Validation { field: String, message: String },
}

fn load_and_parse(path: &str) -> Result<i32, AppError> {
    let content = std::fs::read_to_string(path)?; // io::Error auto-converted
    let number: i32 = content.trim().parse()?;   // ParseIntError auto-converted
    Ok(number)
}
```

2.5 Pattern Matching

Rust's match is exhaustive — the compiler forces you to handle every possible case. It is far more powerful than a switch statement.

```
// Basic match
let number = 7;
match number {
    1      => println!("One"),
    2 | 3  => println!("Two or three"), // multiple patterns
    4..=6  => println!("Four to six"),  // range pattern
}
```

```

        n          => println!("Something else: {n}"), // binding
    }

    // Match on enums – the primary use case
    #[derive(Debug)]
    enum Message {
        Quit,
        Move { x: i32, y: i32 },
        Write(String),
        ChangeColour(u8, u8, u8),
    }

    fn process(msg: Message) {
        match msg {
            Message::Quit          => println!("Quitting"),
            Message::Move { x, y } => println!("Move to ({x}, {y})"),
            Message::Write(text)    => println!("Text: {text}"),
            Message::ChangeColour(r, g, b) => println!("Colour: {r},{g},{b}"),
        }
    }

    // Match guards – add conditions to a pattern arm
    let pair = (2, -2);
    match pair {
        (x, y) if x == y          => println!("Equal"),
        (x, y) if x + y == 0       => println!("Sum to zero"),
        (x, _) if x % 2 == 0       => println!("First is even"),
        _                          => println!("Other"),
    }

    // Destructuring in let, for, function params
    let (a, b, c) = (1, 2, 3);
    let Point { x, y } = Point { x: 5, y: 10 };

    struct Point { x: i32, y: i32 }

```

2.6 Traits

Traits define shared behaviour. A trait is a collection of method signatures that a type can implement. They are Rust's primary mechanism for polymorphism.

```

// Define a trait
trait Summary {
    fn summarise(&self) -> String;

    // Default implementation – types can override or inherit
    fn preview(&self) -> String {
        format!("{}", ..50.min(self.summarise().len()))
    }
}

struct Article {

```

```

        title: String,
        content: String,
    }

    struct Tweet {
        username: String,
        content: String,
    }

    // Implement the trait for each type
    impl Summary for Article {
        fn summarise(&self) -> String {
            format!("{}: {}", self.title, self.content)
        }
    }

    impl Summary for Tweet {
        fn summarise(&self) -> String {
            format!("{}: {}", self.username, self.content)
        }
    }

    // Trait bounds – accept any type that implements Summary
    fn notify(item: &impl Summary) {
        println!("Breaking news! {}", item.summarise());
    }

    // Equivalent using full where clause syntax
    fn notify_where<T>(item: &T) where T: Summary {
        println!("Breaking news! {}", item.summarise());
    }

    // Multiple trait bounds
    fn display_and_summarise<T: Summary + std::fmt::Display>(item: &T) {
        println!("{}", item); // uses Display
        println!("{}", item.summarise()); // uses Summary
    }

```

Common Standard Library Traits

```

// Display – human-readable formatting with {}
use std::fmt;
struct Point { x: f64, y: f64 }
impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

// Debug – debug formatting with {:?} – usually derived
#[derive(Debug)]
struct Rect { w: f64, h: f64 }

// Clone – explicit deep copy
#[derive(Clone, Debug)]
struct Config { name: String, value: i32 }
let c1 = Config { name: String::from("x"), value: 10 };

```

```
let c2 = c1.clone(); // c1 is still valid

// PartialEq / Eq - equality comparisons
#[derive(PartialEq, Debug)]
struct Colour(u8, u8, u8);
let red = Colour(255, 0, 0);
assert_eq!(red, Colour(255, 0, 0));

// PartialOrd / Ord - comparison / sorting
// Iterator - implement next() to make a type iterable
// From / Into - type conversions
```

2.7 Generics

Generics let you write code that works for multiple types without duplicating it. Unlike runtime generics (like Java's), Rust generics are monomorphised — the compiler generates specific code for each type used. Zero runtime cost.

```
// Generic function
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest { largest = item; }
    }
    largest
}

let numbers = vec![34, 50, 25, 100, 65];
let chars   = vec!['y', 'm', 'a', 'q'];
println!("{}", largest(&numbers)); // 100
println!("{}", largest(&chars));   // y

// Generic struct
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

impl<T> Pair<T> {
    fn new(first: T, second: T) -> Self { Self { first, second } }
}

// Conditionally implement methods based on trait bounds
impl<T: std::fmt::Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.first >= self.second {
            println!("Largest is first: {}", self.first);
        } else {
            println!("Largest is second: {}", self.second);
        }
    }
}
```

```
}

// Generic enum - how Option and Result are defined
enum MyOption<T>          { Some(T), None }
enum MyResult<T, E>       { Ok(T), Err(E) }
```


PHASE 3

Ownership Deep Dive

3.1 Lifetimes

Lifetimes are how Rust ensures that references are always valid. The compiler tracks how long references live to prevent dangling pointers. Most of the time, lifetimes are inferred — but when functions take and return references, you may need to annotate them explicitly.

```
// The problem lifetimes solve: dangling references
// fn longest(x: &str, y: &str) -> &str {
//     if x.len() > y.len() { x } else { y }
// }
// ERROR: Rust doesn't know if the return value lives as long as x or y

// Solution: annotate with a lifetime parameter 'a
// 'a means: the returned &str will live at least as long as both x and y
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("long string");
    let result;
    {
        let s2 = String::from("xyz");
        result = longest(s1.as_str(), s2.as_str());
        println!("Longest: {result}"); // OK - result used while s2 is alive
    }
    // println!("{result}"); // ERROR - s2 dropped, result might dangle
}

// Lifetime in structs
// This struct cannot outlive the &str it holds
struct Excerpt<'a> {
    text: &'a str,
}

let novel = String::from("Call me Ishmael. Some years ago...");
let first_sentence = novel.split('.').next().unwrap();
let excerpt = Excerpt { text: first_sentence };
// excerpt cannot outlive `novel`
```

Lifetime Elision

Rust has three lifetime elision rules that allow you to omit lifetime annotations in common patterns. Most single-parameter functions returning a reference don't need annotations. The compiler infers them automatically.

```
// The 'static lifetime — lives for the entire program duration
// String literals have 'static lifetime (stored in binary)
let s: &'static str = "I have a static lifetime";

// Generic type parameters + trait bounds + lifetimes together
fn longest_with_announcement<'a, T>(
    x:          &'a str,
    y:          &'a str,
    announcement: T,
) -> &'a str
where
    T: std::fmt::Display,
{
    println!("Announcement: {announcement}");
    if x.len() > y.len() { x } else { y }
}
```

3.2 Smart Pointers

Smart pointers are data structures that act like pointers but have additional metadata and capabilities. They manage heap allocation and handle memory safety in scenarios the basic borrow checker can't cover.

Box<T> — Heap Allocation

```
// Box<T> puts a value on the heap
let b = Box::new(5); // 5 is on the heap, b is on the stack
println!("{b}");    // 5 — Box auto-derefs

// Primary use case: recursive types (which would be infinitely sized on stack)
#[derive(Debug)]
enum List {
    Cons(i32, Box<List>), // Box breaks the infinite size
    Nil,
}

use List::{Cons, Nil};
let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
println!("{:?}", list);

// Box also used to store trait objects (dynamic dispatch)
trait Animal { fn sound(&self) -> &str; }
struct Dog;
struct Cat;
impl Animal for Dog { fn sound(&self) -> &str { "woof" } }
impl Animal for Cat { fn sound(&self) -> &str { "meow" } }

let animals: Vec<Box<dyn Animal>> = vec![Box::new(Dog), Box::new(Cat)];
for a in &animals { println!("{}", a.sound()); }
```

Rc<T> — Reference Counting (Single Thread)

```

use std::rc::Rc;

// Rc<T> allows multiple owners – reference count tracks how many
// Cannot be shared across threads
let a = Rc::new(String::from("shared data"));
let b = Rc::clone(&a); // increment ref count – NOT a deep copy
let c = Rc::clone(&a);

println!("Count: {}", Rc::strong_count(&a)); // 3
println!("{}", a); // shared data

drop(c); // decrement count
println!("Count: {}", Rc::strong_count(&a)); // 2
// Data is freed when count reaches 0

// Weak<T> – non-owning reference to break cycles
use std::rc::Weak;

let strong = Rc::new(String::from("hello"));
let weak: Weak<String> = Rc::downgrade(&strong);

// Weak must be upgraded to access the value
if let Some(val) = weak.upgrade() {
    println!("{}", val); // only runs if value still exists
}
drop(strong);
assert!(weak.upgrade().is_none()); // value was dropped

```

RefCell<T> — Interior Mutability

```

use std::cell::RefCell;

// RefCell<T> enforces borrow rules at RUNTIME instead of compile time
// Allows mutation through a shared (&) reference

let data = RefCell::new(vec![1, 2, 3]);

// Borrow immutably
{
    let v = data.borrow(); // returns Ref<Vec<i32>>
    println!("{:?}", *v); // [1, 2, 3]
} // immutable borrow ends here

// Borrow mutably
{
    let mut v = data.borrow_mut(); // returns RefMut<Vec<i32>>
    v.push(4);
} // mutable borrow ends here

println!("{:?}", data.borrow()); // [1, 2, 3, 4]

// Common pattern: Rc<RefCell<T>> – shared ownership with mutation
use std::rc::Rc;
use std::cell::RefCell;

```

```
#[derive(Debug)]
struct Node { value: i32, children: Vec<Rc<RefCell<Node>>> }

let root = Rc::new(RefCell::new(Node { value: 1, children: vec![] }));
let child = Rc::new(RefCell::new(Node { value: 2, children: vec![] }));
root.borrow_mut().children.push(Rc::clone(&child));
```

Arc<T> + Mutex<T> — Thread-Safe Shared State

```
use std::sync::{Arc, Mutex};
use std::thread;

// Arc = Atomic Rc — reference counting safe across threads
// Mutex = Mutual exclusion — only one thread accesses at a time

let counter = Arc::new(Mutex::new(0));
let mut handles = vec![];

for _ in 0..10 {
    let counter = Arc::clone(&counter);
    let handle = thread::spawn(move || {
        let mut num = counter.lock().unwrap(); // acquire the lock
        *num += 1;
    }); // lock released when `num` goes out of scope
    handles.push(handle);
}

for h in handles { h.join().unwrap(); }
println!("Result: {}", *counter.lock().unwrap()); // 10
```

PHASE 4

Iterators, Closures & Collections

4.1 Closures

Closures are anonymous functions that can capture variables from their surrounding scope. They are the building blocks of Rust's iterator system.

```
// Basic closure syntax - |params| body
let add_one = |x| x + 1;
let add = |x, y| x + y;
let print = |s: &str| println!("{s}");

println!("{}", add_one(5)); // 6
println!("{}", add(3, 4)); // 7

// Closures CAPTURE their environment
let threshold = 10;
let is_big = |n: i32| n > threshold; // captures threshold by reference
println!("{}", is_big(15)); // true
println!("{}", is_big(5)); // false

// move closure - takes ownership of captured variables
// Required when the closure outlives the scope of the captured data
let name = String::from("Alice");
let greet = move || println!("Hello, {name}"); // name is moved into closure
// println!("{name}"); // error - name was moved
greet(); // Hello, Alice

// The three Fn traits:
// Fn      - borrows immutably; can be called multiple times
// FnMut   - borrows mutably; can be called multiple times
// FnOnce  - takes ownership; can only be called once

fn apply<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }
fn apply_mut<F: FnMut() -> String>(mut f: F) -> String { f() }
fn apply_once<F: FnOnce() -> String>(f: F) -> String { f() }

let result = apply(|x| x * x, 5); // 25
let mut count = 0;
let mut counter = || { count += 1; count }; // FnMut
println!("{}", counter(), counter()); // 1 2
```

4.2 Iterators

Rust's iterators are lazy and composable. Chains of iterator operations compile to tight loops with no overhead — the zero-cost abstraction principle at work.

```
// The Iterator trait — implement next() to be iterable
// trait Iterator { type Item; fn next(&mut self) -> Option<Self::Item>; }

let v = vec![1, 2, 3, 4, 5];

// Three ways to iterate a Vec
for x in &v          { /* &i32 — borrow */ }
for x in &mut v      { /* &mut i32 — mutable borrow */ }
// (requires: let mut v)
for x in v          { /* i32 — consumed, v is moved */ }

// Iterator adaptors — lazy transformations (nothing executes yet)
let v = vec![1, 2, 3, 4, 5, 6];

let evens_doubled: Vec<i32> = v.iter()
    .filter(|&x| x % 2 == 0)    // keep evens: [2, 4, 6]
    .map(|&x| x * 2)           // double: [4, 8, 12]
    .collect();                // consume and build Vec
println!("{:?}", evens_doubled); // [4, 8, 12]

// fold / reduce — accumulate a single value
let sum: i32 = (1..=100).fold(0, |acc, x| acc + x);
println!("{sum}"); // 5050

// Common adaptors
let nums = vec![3, 1, 4, 1, 5, 9, 2, 6];
nums.iter().count();           // 8
nums.iter().sum::<i32>();       // 31
nums.iter().max();              // Some(9)
nums.iter().min();              // Some(1)
nums.iter().any(|&x| x > 8);    // true
nums.iter().all(|&x| x > 0);    // true
nums.iter().find(|&x| x > 4);   // Some(5)
nums.iter().position(|&x| x == 5); // Some(4)
nums.iter().enumerate();        // yields (index, &value)
nums.iter().zip(nums.iter());   // pairs elements from two iterators
nums.iter().chain(nums.iter()); // concatenate two iterators
nums.iter().take(3);            // first 3: [3, 1, 4]
nums.iter().skip(3);            // skip first 3: [1, 5, 9, 2, 6]
nums.iter().peekable();         // peek at next without consuming
nums.iter().flat_map(|&x| 0..x); // flatten nested iterators
nums.iter().flatten();          // for iterators of iterators
nums.iter().cloned();           // &T -> T (clone each element)
nums.iter().copied();           // &T -> T (copy each — for Copy types)
```

Building Custom Iterators

```
// Implement Iterator for your own type
struct Counter { count: u32, max: u32 }

impl Counter {
    fn new(max: u32) -> Self { Counter { count: 0, max } }
```

```

}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        if self.count < self.max {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

// Once Iterator is implemented, ALL adaptor methods are available for free
let sum: u32 = Counter::new(5).sum(); // 15
let pairs: Vec<_> = Counter::new(5).zip(Counter::new(5).skip(1)).collect();
// [(1,2), (2,3), (3,4), (4,5)]

```

4.3 Collections

Vec<T> — Dynamic Array

```

// Creating Vecs
let v: Vec<i32> = Vec::new();
let v = vec![1, 2, 3]; // macro
let v: Vec<i32> = (1..=5).collect(); // from iterator

// Modifying
let mut v = vec![1, 2, 3];
v.push(4); // append: [1,2,3,4]
v.pop(); // remove last: Some(4), v=[1,2,3]
v.insert(1, 10); // insert at index: [1,10,2,3]
v.remove(1); // remove at index: [1,2,3]
v.retain(|&x| x > 1); // keep matching: [2,3]
v.sort(); // sort in place
v.dedup(); // remove consecutive duplicates
v.reverse(); // reverse in place
v.extend([4, 5, 6]); // append from iterator

// Accessing
let v = vec![10, 20, 30];
v[1]; // 20 - panics if out of bounds
v.get(1); // Some(20) - safe, returns Option
v.get(99); // None
v.first(); // Some(10)
v.last(); // Some(30)
v.len(); // 3
v.is_empty(); // false
v.contains(&20); // true

// Slicing
let slice: &[i32] = &v[1..]; // [20, 30]

```

HashMap<K, V>

```

use std::collections::HashMap;

let mut scores: HashMap<String, i32> = HashMap::new();

// Insert
scores.insert(String::from("Alice"), 10);
scores.insert(String::from("Bob"), 20);

// Access
scores.get("Alice");           // Some(10)
scores["Alice"];               // 10 - panics if missing
scores.contains_key("Bob");    // true

// The entry API - the idiomatic way to insert-or-update
scores.entry(String::from("Alice")).or_insert(0); // no-op - Alice exists
scores.entry(String::from("Carol")).or_insert(50); // inserts Carol: 50

// Update based on existing value
let text = "hello world hello rust hello";
let mut word_count: HashMap<&str, i32> = HashMap::new();
for word in text.split_whitespace() {
    let count = word_count.entry(word).or_insert(0);
    *count += 1;
}
// { "hello": 3, "world": 1, "rust": 1 }

// Iterating
for (key, value) in &scores {
    println!("{key}: {value}");
}

// Building from iterator of tuples
let map: HashMap<_, _> = vec![("a", 1), ("b", 2)].into_iter().collect();

```

String — Owned UTF-8 Text

```

// String vs &str
// &str = borrowed string slice = view into string data (no ownership)
// String = owned, heap-allocated, growable UTF-8 text

// Creating Strings
let s = String::new();
let s = String::from("hello");
let s = "hello".to_string();
let s = format!("{}", "hello", "world"); // format! is very common

// Modifying
let mut s = String::from("hello");
s.push_str(", world"); // append &str
s.push('!');           // append char
let s3 = s + &s;       // + moves the left operand - can't use s after this

```



```
// Slicing - slices by BYTES, not chars - be careful with Unicode
let hello = "Здравствуйтe"; // Russian - each char is 2 bytes
let s = &hello[0..4];        // first 4 bytes = 'Зд' (2 Cyrillic chars)

// Iterating correctly over characters
for c in "hello".chars() { print!("{c} "); } // h e l l o
for b in "hello".bytes() { print!("{b} "); } // 104 101 108 108 111
```

PHASE 5

Advanced Rust

5.1 Advanced Traits

Trait Objects — Dynamic Dispatch with dyn

```
// Static dispatch (generics) – compiler generates specific code for each type
// Fast, but all types must be known at compile time
fn notify_static<T: Summary>(item: &T) { /* ... */ }

// Dynamic dispatch (trait objects) – resolved at runtime
// Use when types aren't known until runtime or you need a heterogeneous
// collection
fn notify_dynamic(item: &dyn Summary) { /* ... */ }

// Box<dyn Trait> – heap-allocated trait object, owned
fn create_summary(is_article: bool) -> Box<dyn Summary> {
    if is_article {
        Box::new(Article { title: String::from("News"), content:
String::from("...") })
    } else {
        Box::new(Tweet { username: String::from("user"), content:
String::from("hi") })
    }
}

// Heterogeneous collection – different concrete types, same trait
let items: Vec<Box<dyn Summary>> = vec![
    Box::new(Article { title: String::from("A"), content: String::from("...") }),
    Box::new(Tweet { username: String::from("u"), content: String::from("t") }),
];
for item in &items { println!("{}", item.summarise()); }

// Object safety rules: a trait can be a trait object only if:
// 1. The trait has no methods that return Self
// 2. The trait has no generic method parameters
```

Associated Types

```
// Associated types: define a type placeholder in the trait
// The implementor specifies the concrete type
trait Container {
    type Item; // associated type
    fn first(&self) -> Option<&Self::Item>;
    fn len(&self) -> usize;
}

struct Stack<T> { items: Vec<T> }
```

```
impl<T> Container for Stack<T> {
    type Item = T; // fill in the associated type
    fn first(&self) -> Option<&T> { self.items.first() }
    fn len(&self)    -> usize      { self.items.len() }
}

// Associated type vs generic trait parameter:
// trait Converter<T> { fn convert(&self) -> T; } // generic: multiple impls per type
// trait Converter    { type Output; fn convert(&self) -> Self::Output; } // one impl

// The Iterator trait uses an associated type for this reason:
// trait Iterator { type Item; fn next(&mut self) -> Option<Self::Item>; }
// Each type can only be an iterator in one way (one Item type)
```

Operator Overloading with std::ops

```
use std::ops::Add;

#[derive(Debug, PartialEq, Clone, Copy)]
struct Vector2D { x: f64, y: f64 }

impl Add for Vector2D {
    type Output = Vector2D;

    fn add(self, other: Vector2D) -> Vector2D {
        Vector2D { x: self.x + other.x, y: self.y + other.y }
    }
}

let v1 = Vector2D { x: 1.0, y: 2.0 };
let v2 = Vector2D { x: 3.0, y: 4.0 };
let v3 = v1 + v2; // calls our Add impl
println!("{:?}", v3); // Vector2D { x: 4.0, y: 6.0 }
```

5.2 Async / Await

Rust's async model is based on Futures — values that represent a computation that hasn't completed yet. The async runtime (usually Tokio) polls futures and drives them to completion.

```
// Add to Cargo.toml:
// tokio = { version = "1", features = ["full"] }
// reqwest = { version = "0.11", features = ["json"] }

use tokio;
```

```
// async fn returns a Future implicitly
#[tokio::main] // sets up the Tokio runtime
async fn main() {
    let result = fetch_data().await;
    println!("{result}");
}

async fn fetch_data() -> String {
    // .await suspends this future until the inner future completes
    // The thread is NOT blocked - it can run other futures
    let response = request::get("https://httpbin.org/get")
        .await
        .expect("Request failed");

    response.text().await.expect("Body failed")
}

// Concurrent execution with tokio::join!
async fn fetch_both() {
    let (r1, r2) = tokio::join!(
        fetch_url("https://example.com/a"),
        fetch_url("https://example.com/b"),
    );
    // Both requests run concurrently - total time = max(time_a, time_b)
}

// Spawn concurrent tasks (like threads, but cooperative)
async fn run_tasks() {
    let h1 = tokio::spawn(async { heavy_computation_1().await });
    let h2 = tokio::spawn(async { heavy_computation_2().await });
    let (r1, r2) = (h1.await.unwrap(), h2.await.unwrap());
}

// Async with error handling
async fn fetch_json<T: serde::de::DeserializeOwned>(url: &str) -> Result<T,
request::Error> {
    request::get(url).await?.json:::<T>().await
}
```

Async Channels

```
use tokio::sync::mpsc;

// Multi-producer, single-consumer async channel
#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel:::<String>(32); // buffer of 32

    // Spawn producer
    tokio::spawn(async move {
        for i in 0..5 {
            tx.send(format!("message {i}")).await.unwrap();
        }
    });

    // Consume
    while let Some(msg) = rx.recv().await {
```

```
        println!("Received: {msg}");
    }
}
```

5.3 Threads & Concurrency

```
use std::thread;
use std::sync::mpsc; // multi-producer single-consumer channel

// Spawning a thread
let handle = thread::spawn(|| {
    for i in 1..=5 {
        println!("Thread: {i}");
    }
});

// Main thread continues...
for i in 1..=3 { println!("Main: {i}"); }

handle.join().unwrap(); // wait for thread to finish

// move closure — thread takes ownership of captured values
let v = vec![1, 2, 3];
let handle = thread::spawn(move || {
    println!("{:?}", v); // v is moved into the thread
});
// v is no longer accessible here
handle.join().unwrap();

// Channels — send data between threads
let (tx, rx) = mpsc::channel();

thread::spawn(move || {
    tx.send(String::from("hi from thread")).unwrap();
    tx.send(String::from("another message")).unwrap();
});

for received in rx { // rx is an iterator
    println!("Got: {received}");
}

// Multiple producers (clone the transmitter)
let (tx, rx) = mpsc::channel();
let tx2 = tx.clone(); // now both tx and tx2 can send
```

5.4 Unsafe Rust

Unsafe Rust lets you step outside the borrow checker's guarantees when you know something it can't prove. The `unsafe` keyword is a signal to future readers that this code requires extra care.

```
// The five things you can do in unsafe that you can't in safe Rust:
// 1. Dereference raw pointers
// 2. Call unsafe functions
// 3. Access or modify a mutable static variable
// 4. Implement an unsafe trait
// 5. Access fields of unions

// Raw pointers - *const T and *mut T
let mut num = 5;
let r1 = &num as *const i32; // raw immutable pointer
let r2 = &mut num as *mut i32; // raw mutable pointer

// Creating raw pointers is safe; dereferencing requires unsafe
unsafe {
    println!("r1 is: {}", *r1);
    println!("r2 is: {}", *r2);
}

// Calling unsafe functions
unsafe fn dangerous() -> i32 { 42 }
let result = unsafe { dangerous() };

// Safe abstraction over unsafe code
// split_at_mut from the standard library (simplified)
fn split_at_mut(slice: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = slice.len();
    let ptr = slice.as_mut_ptr();
    assert!(mid <= len);
    unsafe {
        (
            std::slice::from_raw_parts_mut(ptr, mid),
            std::slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}

// extern - call C code from Rust
extern "C" {
    fn abs(input: i32) -> i32;
}
let result = unsafe { abs(-3) }; // 3
```

5.5 Macros

Macros are code that generates code. They operate on the abstract syntax tree before compilation. Rust has two kinds: declarative macros (`macro_rules!`) and procedural macros (`#[derive(...)]`).

Declarative Macros — macro_rules!

```
// macro_rules! matches patterns and expands them
macro_rules! say_hello {
    () => { println!("Hello!"); };
    ($name:expr) => { println!("Hello, {}!", $name); };
}

say_hello!();           // Hello!
say_hello!("Alice");    // Hello, Alice!

// A simplified vec! macro
macro_rules! my_vec {
    ( $( $x:expr ),* ) => { // $(...),* means 'zero or more comma-separated
    exprs'
        {
            let mut v = Vec::new();
            $( v.push($x); )* // repeat for each matched expr
            v
        }
    };
}

let v = my_vec![1, 2, 3]; // same as vec![1, 2, 3]

// Useful built-in macros
println!("formatted {}", value); // print with newline
format!("returns a {}", "String"); // format to String
eprintln!("error output"); // print to stderr
assert!(condition); // panic if false
assert_eq!(a, b); // panic if a != b
panic!("something went wrong"); // unconditional panic
todo!(); // marks unimplemented code
unimplemented!(); // like todo! but stronger
dbg!(&value); // print value with file/line info
include_str!("file.txt"); // embed file as &str at compile time
```

Derive Macros — Automatic Trait Implementation

```
// #[derive(...)] generates trait implementations automatically
#[derive(
    Debug,          // {:?} formatting
    Clone,          // .clone() method
    PartialEq,      // == and != operators
    Eq,             // total equality (requires PartialEq)
    Hash,           // use as HashMap key
    PartialOrd,     // <, >, <=, >= operators
    Ord,            // total ordering (requires PartialOrd)
    Default,        // Config::default() - all fields get their Default value
)]
struct Config {
    name: String,
    timeout: u32,
    debug: bool,
}

// serde derive macros - Serialize and Deserialize
// Cargo.toml: serde = { version = "1", features = ["derive"] }
```

```
//             serde_json = "1"
use serde::{Serialize, Deserialize};

#[derive(Debug, Serialize, Deserialize)]
struct User {
    id:    u32,
    name:  String,
    email: String,
}

let user = User { id: 1, name: String::from("Alice"), email:
String::from("a@b.com") };
let json = serde_json::to_string(&user).unwrap();
// {"id":1,"name":"Alice","email":"a@b.com"}

let user2: User = serde_json::from_str(&json).unwrap();
```


PHASE 6

Ecosystem & Real-World Patterns

6.1 Cargo In Depth

```
# Cargo.toml - project manifest
[package]
name      = "my-project"
version   = "0.1.0"
edition   = "2021"

[dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
reqwest = "0.11"

[dev-dependencies] # only for tests
proptest = "1"

[features] # optional feature flags
default   = ["json"]
json      = ["dep:serde_json"]
database  = ["dep:sqlx"]

[profile.release] # release build optimisations
opt-level = 3
lto       = true
strip     = true   # strip debug symbols from binary
```

```
# Common Cargo commands
cargo new my-project           # new binary project
cargo new my-lib --lib         # new library project
cargo build                    # debug build
cargo build --release           # optimised release build
cargo run                      # build and run
cargo run -- --arg value       # run with arguments
cargo test                     # run all tests
cargo test test_name           # run tests matching name
cargo test -- --nocapture      # show println! output in tests
cargo clippy                   # linter - catches common mistakes
cargo fmt                      # format all code with rustfmt
cargo doc --open                # build and open documentation
cargo check                    # type-check without building (fast)
cargo add serde                 # add a dependency
cargo update                   # update dependencies
cargo tree                     # show dependency tree
cargo bench                    # run benchmarks
cargo publish                   # publish to crates.io
```

6.2 Testing

```
// Unit tests – in the same file as the code
pub fn add(a: i32, b: i32) -> i32 { a + b }

pub fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 { None } else { Some(a / b) }
}

#[cfg(test)] // only compiled when running tests
mod tests {
    use super::*; // import everything from parent module

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
        assert_eq!(add(-1, 1), 0);
    }

    #[test]
    fn test_divide_by_zero() {
        assert!(divide(10.0, 0.0).is_none());
    }

    #[test]
    fn test_divide() {
        let result = divide(10.0, 4.0).unwrap();
        assert!((result - 2.5).abs() < f64::EPSILON);
    }

    #[test]
    #[should_panic(expected = "divide by zero")]
    fn test_panics() {
        panic!("divide by zero");
    }

    #[test]
    #[ignore] // skip this test unless: cargo test -- --ignored
    fn expensive_test() { /* ... */ }
}

// Integration tests – in tests/ directory
// tests/integration_test.rs
use my_crate::add;

#[test]
fn integration_add() {
    assert_eq!(add(2, 2), 4);
}

// Doc tests – in documentation comments
/// Adds two numbers.
///
/// # Examples
/// ```
/// let result = my_crate::add(2, 3);
/// assert_eq!(result, 5);
/// ```
```

```
/// ```
pub fn add(a: i32, b: i32) -> i32 { a + b }
```

6.3 Serde — Serialisation

```
use serde::{Serialize, Deserialize};
use serde_json;

#[derive(Debug, Serialize, Deserialize, PartialEq)]
struct Config {
    #[serde(rename = "app_name")] // rename field in JSON
    name: String,

    #[serde(default)] // use Default if missing from JSON
    timeout: u32,

    #[serde(skip_serializing_if = "Option::is_none")] // omit if None
    description: Option<String>,
}

// Serialise to JSON string
let config = Config {
    name: String::from("my-app"),
    timeout: 30,
    description: Some(String::from("A great app")),
};
let json = serde_json::to_string_pretty(&config).unwrap();
println!("{}", json);
// {
//   "app_name": "my-app",
//   "timeout": 30,
//   "description": "A great app"
// }

// Deserialise from JSON
let json_str = r#"{"app_name": "test"}"#;
let parsed: Config = serde_json::from_str(json_str).unwrap();
// timeout = 0 (default), description = None

// serde works with many formats: JSON, TOML, YAML, MessagePack, bincode...
// toml = "0.8"
let toml_str = toml::to_string(&config).unwrap();
let from_toml: Config = toml::from_str(&toml_str).unwrap();
```

6.4 Building a Real CLI Tool with Clap

```
// Cargo.toml: clap = { version = "4", features = ["derive"] }
```

```
use clap::{Parser, Subcommand};

#[derive(Parser, Debug)]
#[command(name = "todo", about = "A simple todo manager", version)]
struct Cli {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand, Debug)]
enum Commands {
    /// Add a new todo item
    Add {
        /// The task description
        #[arg(short, long)]
        task: String,

        /// Priority level (1-5)
        #[arg(short, long, default_value_t = 1)]
        priority: u8,
    },

    /// List all todo items
    List {
        #[arg(short, long, help = "Show completed items too")]
        all: bool,
    },

    /// Mark an item as done
    Done {
        #[arg(help = "ID of the item to complete")]
        id: u32,
    },
}

fn main() {
    let cli = Cli::parse();
    match cli.command {
        Commands::Add { task, priority } => {
            println!("Adding: '{task}' with priority {priority}");
        }
        Commands::List { all } => {
            println!("Listing todos (include_done: {all})");
        }
        Commands::Done { id } => {
            println!("Marking {id} as done");
        }
    }
}

// Auto-generated --help output:
// Usage: todo <COMMAND>
// Commands:
//   add    Add a new todo item
//   list   List all todo items
//   done   Mark an item as done
```

6.5 Axum — Async Web API

```
// Cargo.toml:
// axum = "0.7"
// tokio = { version = "1", features = ["full"] }
// serde = { version = "1", features = ["derive"] }

use axum::{
    routing::{get, post},
    extract::{Path, State, Json},
    response::IntoResponse,
    Router,
};
use serde::{Deserialize, Serialize};
use std::sync::{Arc, Mutex};
use std::collections::HashMap;

#[derive(Debug, Clone, Serialize, Deserialize)]
struct User { id: u32, name: String }

#[derive(Deserialize)]
struct CreateUser { name: String }

type SharedState = Arc<Mutex<HashMap<u32, User>>>>;

#[tokio::main]
async fn main() {
    let state: SharedState = Arc::new(Mutex::new(HashMap::new()));

    let app = Router::new()
        .route("/users", get(list_users).post(create_user))
        .route("/users/{id}", get(get_user))
        .with_state(state);

    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    println!("Listening on :3000");
    axum::serve(listener, app).await.unwrap();
}

async fn list_users(State(state): State<SharedState>) -> impl IntoResponse {
    let users: Vec<User> = state.lock().unwrap().values().cloned().collect();
    Json(users)
}

async fn create_user(
    State(state): State<SharedState>,
    Json(payload): Json<CreateUser>,
) -> impl IntoResponse {
    let mut db = state.lock().unwrap();
    let id = db.len() as u32 + 1;
    let user = User { id, name: payload.name };
    db.insert(id, user.clone());
    (axum::http::StatusCode::CREATED, Json(user))
}

async fn get_user(
    Path(id): Path<u32>,

```

```
State(state): State<SharedState>,
) -> impl IntoResponse {
    match state.lock().unwrap().get(&id).cloned() {
        Some(user) => (axum::http::StatusCode::OK, Json(Some(user))),
        None       => (axum::http::StatusCode::NOT_FOUND, Json(None)),
    }
}
```

You've reached the end of the Rust coursework.

Now fight the borrow checker. It is the best teacher in programming.